DERİN ÖĞRENME · KARŞILAŞTIRMALI ANALİZ

S&P 500 Yön Tahmini

Kendi Feedforward Projem ile
Samuel Edet'in RNN Çalışmasının Kıyaslanması

MIT 6.S191 prensipleri üzerine inşa edilen ileri beslemeli sinir ağımı, akademik literatürdeki tekrarlayan sinir ağları (RNN / LSTM / GRU) temelli bir çalışma ile karşılaştıran; her iki yaklaşımın veri hazırlığı, mimari, eğitim ve performans boyutlarında hangi derslerin alınabileceğini inceleyen teknik bir rapor.

Feedforward Dense Model · Binary Classification · TensorFlow / Keras · RNN · LSTM · GRU

MIT 6.S191 · INTRODUCTİON TO DEEP LEARNİNG · FEEDFORWARD BASELİNE

1. Giriş: S&P 500'ü Neden Tahmin Etmeye Çalışıyoruz?

S&P 500, Amerika Birleşik Devletleri'nin en büyük 500 halka açık şirketinin performansını tek bir rakamda özetleyen bir endekstir. Apple, Microsoft, Coca-Cola, JPMorgan… Aklınıza gelebilecek tüm büyük markalar bu sepetin içindedir. Bu yüzden S&P 500, yalnızca bir sayı değil; dünya ekonomisinin nabzını tutan bir **termometredir**. Endeks yükseldiğinde küresel iyimserlik, düştüğünde ise tedirginlik hakimdir.

Peki bu "termometreyi" önceden okumak neden bu kadar zordur?

Finansal piyasalar, teknik tabirle **düşük sinyal-gürültü oranına (low signal-to-noise ratio)** sahip sistemlerdir. Bunu şöyle hayal edebilirsiniz: Kalabalık bir konser salonunda arkadaşınızın size bir şey fısıldamaya çalıştığını düşünün. Ortamda bir *"sinyal"* (arkadaşınızın sesi) ve bir yığın *"gürültü"* (müzik, kalabalık, bağırışlar) vardır. Finansal piyasalarda da durum böyledir: Fiyatları yönlendiren gerçek ekonomik nedenler (şirket karlılığı, faiz kararları, jeopolitik olaylar) vardır, ama bunların üzerine milyonlarca yatırımcının psikolojik tepkileri, panik satışları, aşırı iyimserlik dalgaları eklenir. Model, bütün bu kakofoninin içinden arkadaşının sesini duymaya çalışır.

İşte **derin öğrenme (deep learning)**, tam da bu noktada sahneye çıkar. Geleneksel istatistiksel modeller (doğrusal regresyon gibi), verinin belli kalıplara uyduğunu varsayar. Derin öğrenme ise bu varsayımları terk edip veriye şunu sorar: *"Söyle bana, hangi kalıpları barındırıyorsun?"* Sinir ağları milyonlarca küçük hesaplamayla verideki doğrusal olmayan (non-linear) örüntüleri keşfedebilir — tıpkı bir dedektifin bir sürü ipucunu birleştirerek katile ulaşması gibi.

Hem benim projemde hem de incelediğim Samuel Edet'in makalesinde amaç aynıdır: **S&P 500'ün yön hareketini (yukarı mı, aşağı mı?) derin öğrenme ile öngörmek.** Ancak iki çalışma, aynı dağa farklı patikalardan tırmanıyor. Bu rapor, o iki patikanın neden ve nasıl farklılaştığını, hangisinin zirveye daha yakın olduğunu ve bir sonraki adımımızın neden Samuel Edet'in patikasına sapmak olması gerektiğini anlatıyor.

2. Kullanılan Modellerin Temel Mantığı

> *Karşılaştırmaya geçmeden önce, iki projenin kullandığı modelleri — bir çocuğa anlatır gibi — açıklamakta fayda var. Çünkü aslında tüm hikâye, modellerin **hafıza sahibi olup olmamasında** saklı.*

2.1 Benim Projem: Feedforward (İleri Beslemeli) Sinir Ağı

Benim modelim **Feedforward Neural Network**, yani *ileri beslemeli sinir ağıdır*. Adını, bilginin içinden yalnızca tek yönde — girdiden çıktıya doğru — akmasından alır. Hiç geriye dönüp eski bilgiye bakmaz.

> 💡 **ANALOJİ**
>
> Feedforward modeli, **sadece bugünün gazetesine bakarak** yarınki havayı tahmin etmeye çalışan bir meteoroloğa benzer. Dün yağmur yağmış mı, hafta başında sıcaklık ne kadarmış, geçen ay kaç fırtına olmuş — bunları hatırlamaz, umursamaz. Önünde sadece o anki veriye bakar ve karar verir.

Modelin omurgası **Dense (Tam-Bağlı) katmanlardan** oluşur. Dense katmanını hayal etmek için bir jüri heyetini düşünün: Her jüri üyesi (nöron), sunumun tamamını dinler, kendi önceliğine göre puan verir ve sonuçları bir sonraki uzmanlar kuruluna iletir. Ben 64 kişilik bir jüriden 32 kişilik bir üst komisyona, oradan da 1 kişilik son karar mercisine bilgi akışı tasarladım.

Benim modelimin mimarisi şöyledir:

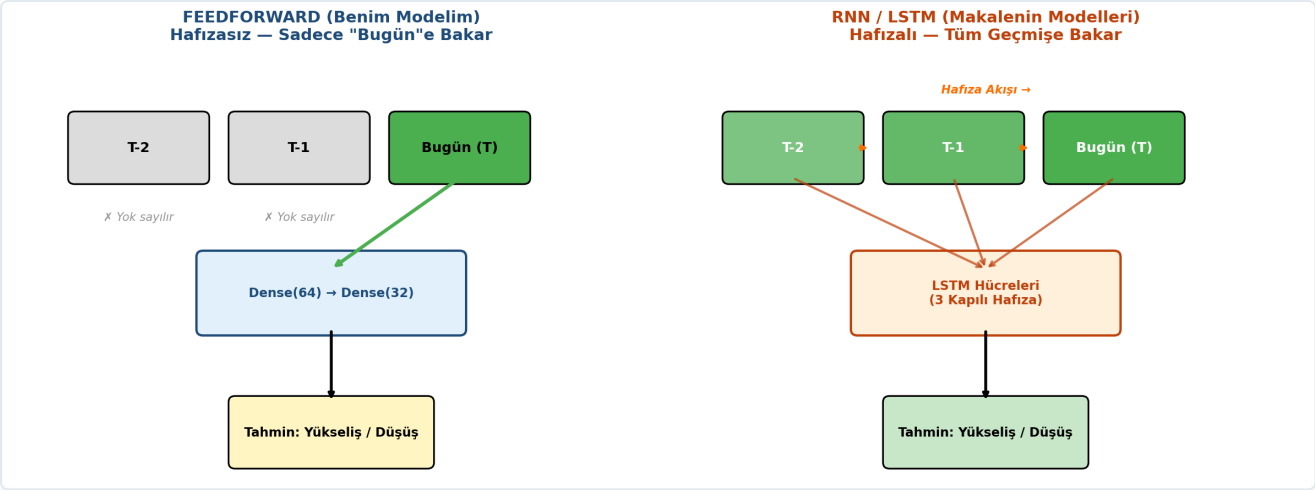
```
Giriş (9 özellik) → Dense(64) → Dropout → Dense(32) → Dropout → Sigmoid
```

Yani 9 adet finansal göstergeyi alıyor, bunları 64 nöronluk bir "jüri heyetinden" geçiriyor, sonra bu kararları 32 nöronluk daha küçük bir "uzmanlar heyetine" taşıyor ve son olarak tek bir nöron 0 ile 1 arası bir olasılık üretiyor: **"Fiyat yükselecek mi?"**

2.2 Makalenin Kalbi: RNN, LSTM ve GRU

Samuel Edet'in makalesi ise hafızalı modelleri kullanıyor. Bunlara genel olarak **Recurrent Neural Networks (RNN) — Tekrarlayan Sinir Ağları** deniyor. Bu modellerin farkı, **zaman içinde hafıza tutabilmeleridir.**

> 💡 **ANALOJİ**
>
> RNN, yarınki havayı tahmin ederken **son bir haftanın gazete arşivini hatırlayan** bir meteorologdur. Bugünkü bulutların yoğunluğunun yanı sıra, son 5 günde basınç nasıl değişmiş, rüzgâr nereden esmiş, önceki fırtına ne kadar sürmüş — hepsini zihnindeki bir "hafıza defterine" kaydeder ve karar verirken bu deftere bakar.



Şekil 1. *Feedforward (hafızasız) ve RNN/LSTM (hafızalı) modellerin girdi işleme mantığındaki temel fark. Feedforward yalnızca anlık veriye bakarken, RNN/LSTM tüm geçmişi zincirleme işler.*

Makalede üç RNN çeşidinden bahsediliyor:

- **Simple RNN (Basit RNN):** En sade hafızalı ağdır. Yeni bilgi geldiğinde hafızayı günceller. Ama problemi şu: Hafıza uzadıkça eski bilgiler silikleşir — buna *"vanishing gradient"* (kaybolan gradyan) denir. Yani "3 hafta önceki düşüş" gibi uzak bilgiler, modelin hafızasında kuma gömülür.
- **LSTM (Long Short-Term Memory — Uzun-Kısa Süreli Hafıza):** Bu problemi çözmek için tasarlanmıştır. LSTM'in içinde üç özel **"kapı"** vardır: *Giriş kapısı* (yeni gelen bilgiyi ne kadar alayım?), *Unutma kapısı* (eski bilgiden neyi unutayım?) ve *Çıkış kapısı* (hafızamdan dışarıya ne göstereyim?). Tıpkı bir öğrencinin sınavdan önce önemli notları saklayıp, gereksizleri eleyip, sadece gerekeni hatırlamasına benzer.
- **GRU (Gated Recurrent Unit):** LSTM'in daha basit ve hızlı akrabasıdır. İki kapılı bir sistemdir — aynı işi LSTM'den daha az parametre ile yapmaya çalışır. Daha az güçlü ama daha hızlı eğitilen bir kardeş gibi düşünün.

Makalede ayrıca **ECNN (Error Correction Neural Network)** de tartışılıyor. Bu model, bir öncekinin hatasını da girdi olarak alıp "bir önceki tahminimde şu kadar yanıldım, bunu düzelteyim" mantığıyla çalışır — özellikle finansal şoklara karşı dayanıklı olması beklenir.

Özellik	Feedforward (Benim)	RNN / LSTM / GRU (Makale)
Hafıza var mı?	❌ Yok	✅ Var
Girdi yapısı	2D: (Örneklem, Özellik)	3D: (Örneklem, Zaman, Özellik)
En iyi olduğu alan	Tabular veriler	Zaman serileri, metin, konuşma
Eğitim hızı	Hızlı	Yavaş (sıralı hesap)
Zaman serisi uygunluğu	Zayıf (baseline)	Güçlü

3. Veri Hazırlığı ve Özellik Seçimi Kıyaslaması

Derin öğrenmede bir söz vardır: **"Çöp girerse, çöp çıkar (Garbage in, garbage out)."** Modelin ne kadar zekice tasarlandığı değil, ona ne kadar temiz ve bilgilendirici veri verdiğiniz başarıyı belirler. İşte tam bu noktada iki proje farklı yollara sapıyor.

3.1 Benim Yaklaşımım: Teknik Göstergelerle Zenginleştirilmiş Tek Kaynak

Ben projemde, yaklaşık **257.000 satırlık** dev bir S&P 500 sektör ETF verisi kullandım (makaleden yaklaşık 120 kat daha fazla örneklem!). Veri setim, sadece ham fiyatı değil, yıllarca trader'ların kullanmaya alışkın olduğu 9 adet **teknik finansal göstergeyi** içeriyor:

- **Log_Getiri:** Günlük yüzde değişimin logaritması — fiyat hareketinin standardize edilmiş hali.
- **Volatilite_10g ve Volatilite_30g:** 10 ve 30 günlük oynaklık — "piyasa ne kadar sinirli?" sorusunun cevabı.
- **RSI_14:** Göreceli Güç Endeksi — "fiyat aşırı mı alındı, aşırı mı satıldı?" diyen bir momentum ölçer.
- **MACD ve bileşenleri:** Hareketli ortalama yakınsama/ıraksama — trendin yönünü ve gücünü ölçer.
- **Bollinger Bantları (BBL / BBU):** Fiyatın istatistiksel olarak "normal aralığının" alt ve üst sınırları.

Bu göstergeleri neden seçtim? Çünkü her biri ham fiyatın içinden farklı bir hikâye çıkarıyor: Volatilite piyasanın duygusunu, RSI momentumu, MACD trendi, Bollinger ise fiyatın ortalamadan ne kadar uzaklaştığını anlatıyor. **Modelin tek bir sayı yerine, finansal piyasanın çoklu bir portresini görmesini sağladım.**

3.2 Makalenin Yaklaşımı: Makro Korelasyon ve Özellik Eleme

Samuel Edet ise çok farklı bir felsefe izliyor. O, S&P 500'e *dışarıdan* bakıyor. "Dünyada olup bitenlerin S&P 500'ü nasıl etkilediğini" anlamaya çalışıyor. 14 değişken kullanıyor:

- S&P 500'ün önceki 3 gün kapanışları (SPYt-1, SPYt-2, SPYt-3)
- **6 farklı ülkenin borsa endeksi** (Hong Kong, Almanya, İngiltere, Fransa, NASDAQ, Dow Jones)
- **5 büyük Amerikan şirketinin hissesi** (Exxon, P&G, GE, Microsoft, Johnson & Johnson)

Burada çok zekice bir nokta var: Makalede yazar, Hong Kong borsasının (HSI) S&P 500 açılmadan *kapanmış* olduğunu vurguluyor. Yani Asya'da olup bitenler, New York'taki seansın yön tayininde işe yarayan **yarın için bir ipucu** niteliği taşıyor. Bu, aynı gün bilgisiyle aynı günü tahmin etme saçmalığından kaçınan, zamansal olarak tutarlı bir seçimdir.

Ancak 14 değişkenin hepsini kullanmak da akıllıca değil. Çünkü bu değişkenlerin bazıları birbirleriyle aşırı **korelasyonlu** — yani "aynı şeyi farklı kelimelerle söylüyorlar". Makale bu problemi **korelasyon eşiği** ile çözüyor: *"İki özellik arasındaki korelasyon 0.5'ten büyükse, sadece daha bilgilendirici olanı tut."* Bu elemenin sonucunda 14 değişkenden geriye yalnızca **3 tanesi** kalıyor: SPYt-1, JNJ (Johnson & Johnson) ve IXIC (NASDAQ).

> 💡 **ANALOJİ**
>
> Bu tıpkı 14 kişilik bir danışma kurulunda 10 kişinin aynı fikirde olduğunu fark edip, sadece en zeki 3 kişiyi masaya oturtmaya benzer. Geri kalan 11 kişi olsa bile ekstra bilgi katmaz, sadece toplantıyı uzatır.

Burada kritik bir not: Makalede özellik seçimi **GPU zamanını optimize etmek** için de yapılıyor. Sinir ağları, GPU üzerinde çalışan pahalı sistemlerdir; 20 deney yapmak ve her birinde 160 epoch'luk eğitim yapmak saatler alabilir. Gereksiz özellikleri elemek, hem modelin doğruluğunu arttırır hem de hesaplama maliyetini düşürür.

3.3 Data Leakage (Veri Sızıntısı) ve Neden `shuffle=False` Kritiktir?

Her iki projede de ortak bir yaklaşım var: **zaman serilerinde veriyi karıştırmamak.**

Data Leakage (Veri Sızıntısı), makine öğrenmesinin en sinsi günahıdır. Kısaca şudur: *Modelin eğitim sırasında, aslında test zamanında elinde olmaması gereken bir bilgiye (örneğin geleceğe ait bir ipucuna) erişmesi.*

> 💡 **ANALOJİ**
>
> Bir öğrenciye sınav sorularını önceden gösterip sonra sınava sokarsanız, o öğrenci %100 alır. Ama gerçekte hiçbir şey öğrenmemiştir. Gerçek hayatta farklı soru gelince çöker. İşte data leakage budur.

Zaman serilerinde data leakage çoğunlukla, veriyi rastgele karıştırıp bölmekten doğar. Eğer 2010-2020 arası veriyi karıştırıp %80'ini eğitim, %20'sini test olarak ayırırsam, modelim 2015'teki bir günü görmüş olabilir ve onunla 2014'teki bir günü tahmin etmeye çalışır. Bu **geleceği kullanarak geçmişi tahmin etmek** demektir — gerçek hayatta imkânsız bir durum.

Çözüm basittir ve ben bunu projemde uyguladım:

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, shuffle=False
)
```

`shuffle=False` parametresi, "veriyi karıştırma, zaman sırasına göre ilk %80'i eğitim, son %20'si test olsun" der. Böylece model, geçmişle eğitilip gelecekle test edilir — tıpkı gerçek hayatta olduğu gibi.

Bunun yanında, özellikleri ölçeklendiren **StandardScaler'ı** sadece eğitim setine `fit` ettim, teste sadece `transform` uyguladım. Neden? Çünkü StandardScaler her özelliğin ortalamasını ve standart sapmasını öğrenir. Eğer bu öğrenme sırasında test verisini de görürse, model dolaylı olarak geleceğin istatistiksel özelliklerini sızdırmış olur. Bu pipeline'a **leakage-safe scaling** denir.

4. Eğitim Süreci ve Karşılaşılan Zorluklar

4.1 Overfitting (Aşırı Öğrenme) — Modelin Ezberci Öğrenciye Dönüşmesi

Derin öğrenmenin en büyük düşmanı **overfitting**tir. Kısaca: *Modelin eğitim verisini ezberlemesi, ama yeni verilere geldiğinde çuvallaması.*

> 💡 **ANALOJİ**
>
> Overfitting, son 10 yılın sınav sorularını ezberlemiş ama konuyu hiç anlamamış bir öğrenciye benzer. Aynı sorular çıkarsa %100 alır. Ama bu yılki sınavda yeni bir soru tipi çıkarsa, perişan olur.

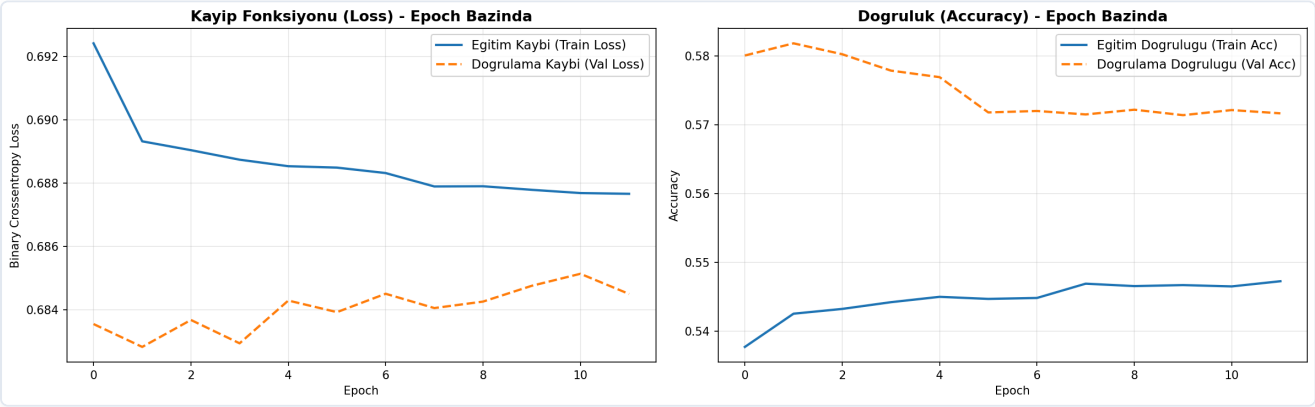
Ben projemde overfitting'e karşı iki silah kullandım:

🛡️ Silah 1: Dropout (%50)

Dropout, eğitim sırasında her adımda nöronların yarısını **rastgele kapatır**. Model bu yüzden hiçbir nörona aşırı bağımlı olamaz; çünkü her an "senin sağ kolunu alıyorum, sol kolunla halletmek zorundasın" diye uyarılır. Tıpkı bir futbol takımının antrenmanda her seferinde farklı bir oyuncuyu eksik oynatması gibi — takım, herkesin rolünü öğrenmek zorunda kalır.

🛡️ Silah 2: Early Stopping (patience=10)

Bu, "doğrulama kaybı 10 epoch boyunca iyileşmezse eğitimi durdur ve en iyi ağırlıkları geri yükle" kuralıdır. Aşçı metaforuyla: Sosu kaynatırken her dakika tadına bakıyorsunuz. En güzel tadı yakaladığınızda, 10 dakika daha bekleyip düzelip düzelmediğine bakıyorsunuz; eğer düzelmiyorsa, ocağı kapatıyorsunuz — pişmişten fazlası yemeği mahveder.



Şekil 2. *Eğitim sürecindeki kayıp ve doğruluk eğrileri. Eğitim ve doğrulama eğrilerinin birbirine yakın seyretmesi, ciddi bir overfitting olmadığını gösteriyor. Modelim 50 epoch yerine 14. epoch'ta Early Stopping ile durmuş.*

Benim modelim, 50 epoch için planlanmıştı ama Early Stopping sayesinde **14. epoch'ta durdu**. Bu iyi bir şey! Demek ki öğrenecek kadarını öğrendi ve daha fazla eğitim sadece ezberciliğe yol açacaktı.

4.2 Makalenin Yaklaşımı: Experimental Design (Deney Tasarımı)

Ben modelimi tek bir mimariye sabitledim: `Dense(64) → Dense(32)`, 50 epoch, Dropout 0.5. Bu klasik ve güvenli bir konfigürasyondur ama sorgulamaya açıktır: "Neden 64 nöron? Neden 32? Neden 50 epoch?"

Samuel Edet bu soruya **Design of Experiment (DoE)** — yani *Deney Tasarımı* yöntemiyle cevap veriyor. Bu yaklaşım, istatistiksel olarak hangi kombinasyonun en iyi sonucu verdiğini sistematik bir şekilde test eder.

Makale şunu yapıyor:

- **Nöron sayısı için 4 seviye** (25, 50, 75, 100)
- **Epoch sayısı için 5 seviye** (40, 80, 120, 160, 200)
- Ve üstüne özellik seçimleri…

Teorik olarak bu $2^{14} \times 4 \times 5$ = **327.680 farklı deney** demektir. Açıkçası kimsenin bunu yapmasına imkân yok. Bu yüzden makale, **fraksiyonel faktöriyel tasarım** ile bu sayıyı 20'ye indiriyor ve her model (Simple RNN, LSTM, GRU) için ayrı ayrı deniyor.

> **Sonuçlar:**
>
> - Simple RNN için en iyi: **75 nöron, 40 epoch → %75 doğruluk**
> - LSTM için: **25 nöron, 80 epoch → %74 doğruluk**
> - GRU için: **50 nöron, 40 epoch → %74 doğruluk**

Ben "hadi şu iyi bilinen mimariyi deneyelim" dedim. Makale ise "hangi mimarinin en iyi çalıştığını sistematik olarak ölçelim" dedi. Bu, amatör futbolcu ile olimpik atlet arasındaki antrenman disiplini farkı gibidir.

5. Performans, Başarı Metrikleri ve Finansal Yorumlama

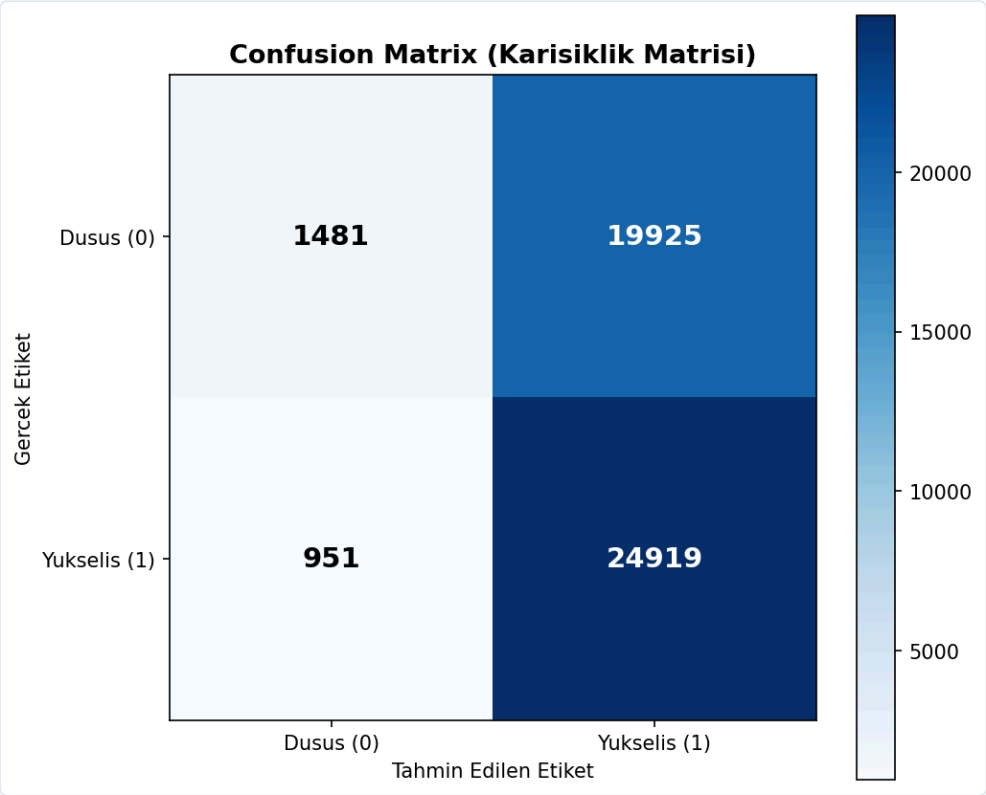
> *Burası hem iki projenin en dramatik farkının ortaya çıktığı hem de en öğretici bölümdür.*

5.1 Benim Sonuçlarım: "Rastgeleliğin Az Üstünde"

TEST DOĞRULUĞU	ROC AUC	EĞİTİM EPOCH
%55.79	**0.514**	**14**
Rastgelenin az üstünde	Sinyal zayıf ama sıfır değil	Early Stopping ile durdu

Bu rakamların ne anlama geldiğini anlamak için önce iki kavramı açıklayayım.

🎯 Confusion Matrix (Karışıklık Matrisi)



Şekil 3. *Modelimin tahmin ile gerçek arasındaki ilişki matrisi. Sağ sütunun baskınlığı (19.925 + 24.919), modelin büyük çoğunlukla "Yükseliş (1)" demeye meyilli olduğunu gösteriyor.*

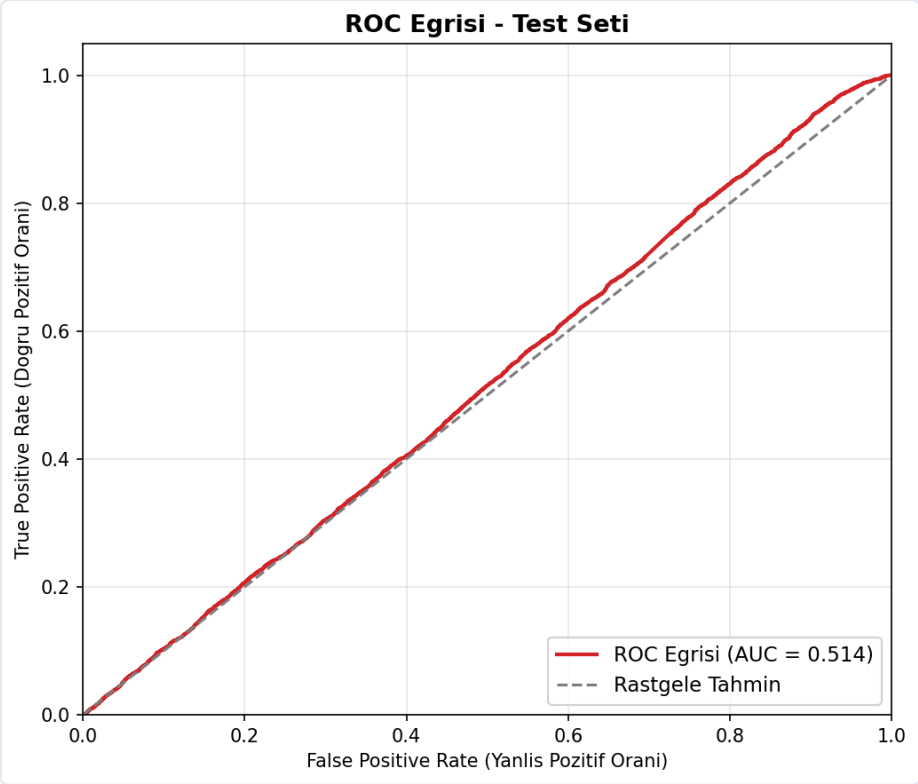
Bu tabloyu bir hakemin maç kararları tablosu gibi düşünün: Her satır gerçeği, her sütun hakemin (modelin) kararını gösterir. Köşegen doğru kararları gösterir.

Dikkat çeken nokta: Benim modelim neredeyse her seferinde **"Yükseliş (1)"** demeye meyilli. Gerçekte düşüş olan 21.406 günün 19.925'inde (!), model yükseliş tahmin etmiş. Bu neden oldu?

İki sebep var:

1. **Sınıf dengesizliği:** Eğitim setimde yükseliş oranı %54.2 — yani veri tabiatı gereği hafifçe yükseliş yönlü. `class_weight` kullanmadığım için model, "hep 1 deyip kabaca doğru çıkmayı" avantaj olarak keşfetti.
2. **Zayıf sinyal:** Modelin gerçekten emin olmadığı durumlarda, "kolay kazanım" olan yükseliş tarafına kaymak doğal davranışı haline geldi.

📈 ROC / AUC Eğrisi

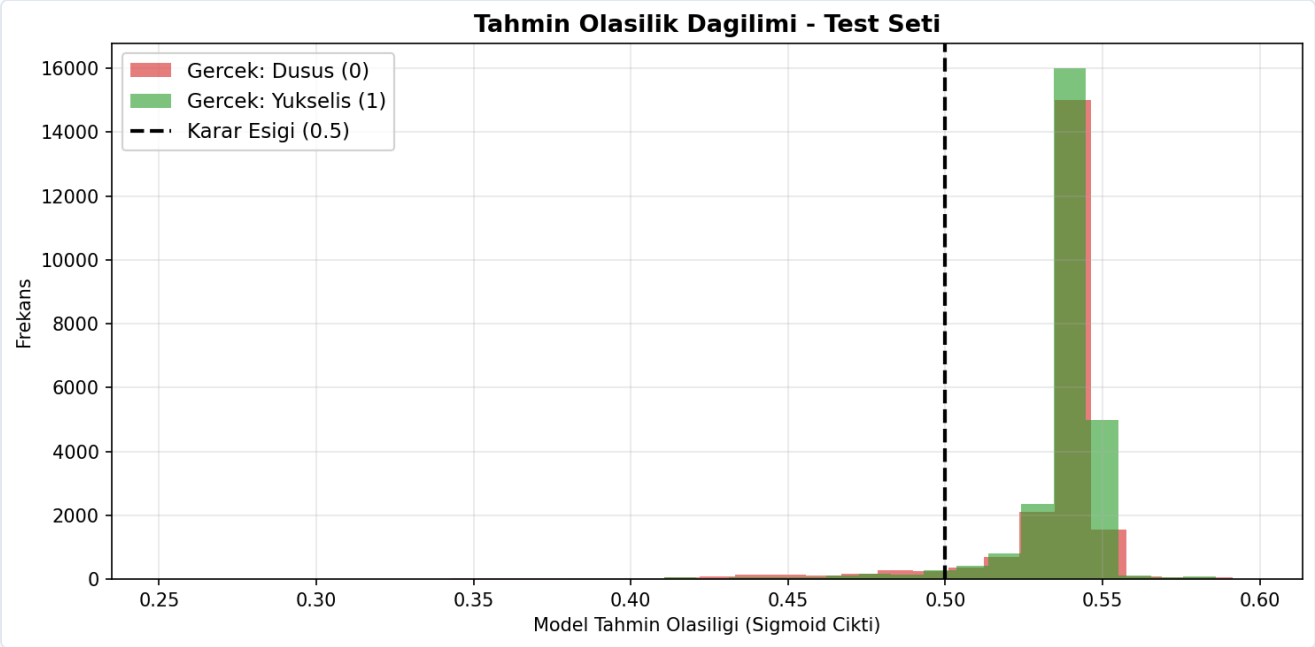


Şekil 4. *Modelimin ROC eğrisi 45° köşegene yapışmış durumda; AUC = 0.514 değeri, modelin yalnızca rastgeleliğin %1.4 üzerinde olduğunu gösteriyor.*

AUC (Area Under Curve), modelin iki sınıfı birbirinden ayırma yeteneğini 0-1 arasında özetleyen bir skordur:

- **AUC = 0.5:** Yazı-tura atmak kadar başarılı (tamamen rastgele).
- **AUC = 1.0:** Kusursuz ayrıştırma (hayal ürünü).
- **AUC = 0.514:** Benim modelim. Rastgeleliğin *yalnızca %1.4 üzerinde*.

5.2 Sigmoid Çıktı Dağılımı: Modelin "Özgüven Eksikliği"



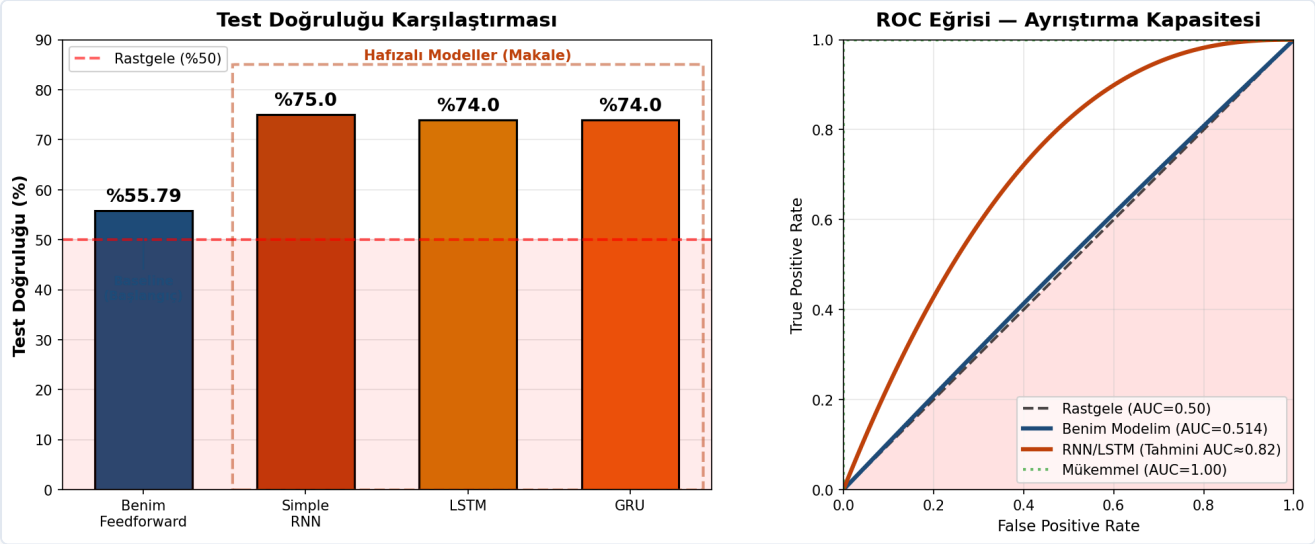
Şekil 5. *Sigmoid olasılıkları 0.50-0.55 aralığında dar bir çan şeklinde sıkışmış. İdeal bir güçlü sınıflandırıcıda bu dağılımın iki kutuplu (0'a ve 1'e yakın) olması beklenirdi.*

> 💡 **ANALOJİ**
>
> Bu tıpkı bir doktorun hiçbir hasta için "kesin sağlıksın" ya da "kesin hastasın" demeyip, herkese "muhtemelen hafif rahatsızsınız, belki öyle belki böyle..." demesine benziyor. Hafif bir kanaati var ama emin değil.

Bu sıkışmanın anlamı: **Model, mevcut özelliklerden elde edebileceği maksimum sinyali çıkarmış durumda — ama bu sinyal, finansal gürültüyü aşacak kadar güçlü değil.**

Pratik sonuç? Karar eşiğini 0.50 yerine 0.55'e çekebilirim — yani "model en az %55 güvenmediği sürece tahmin yapmasın" diyebilirim. Bu, tahmin sayısını azaltır ama her bir tahminin güvenilirliğini artırır.

5.3 Makalenin Sonuçları ve Aradaki Devasa Uçurum



Şekil 6. *Solda: Test doğruluğu karşılaştırması. Benim %55.79 vs makalenin %74-75 skorları arasındaki ~19 puanlık uçurum. Sağda: ROC eğrilerinin ayrıştırma kapasitesi — mavi benim modelim, turuncu ise tahmini RNN performansı.*

Aradaki fark **devasa**: Ben %55.79, o %75. Yaklaşık **19 puanlık** bir uçurum. Peki bu fark nereden geliyor?

> **Kritik cevap: Zamansal hafıza.** Benim modelim, bir günü tek başına değerlendiriyor; *"Bugün RSI 45, volatilite 0.012, MACD pozitif… Yani?"* RNN/LSTM ise sorunu şöyle formüle ediyor: *"Son 5 günde RSI 30'dan 45'e yükseldi, volatilite azalıyor, MACD güç kazanıyor… İşte bu pattern bir yükseliş habercisi, çünkü son 10 yılda benzer pattern'ler 74 kere yükselişle sonuçlandı."*

RNN/LSTM, bir futbol maçını tek bir fotoğraf karesi yerine 90 dakikalık video olarak izliyor. Doğal olarak çok daha iyi yorumluyor.

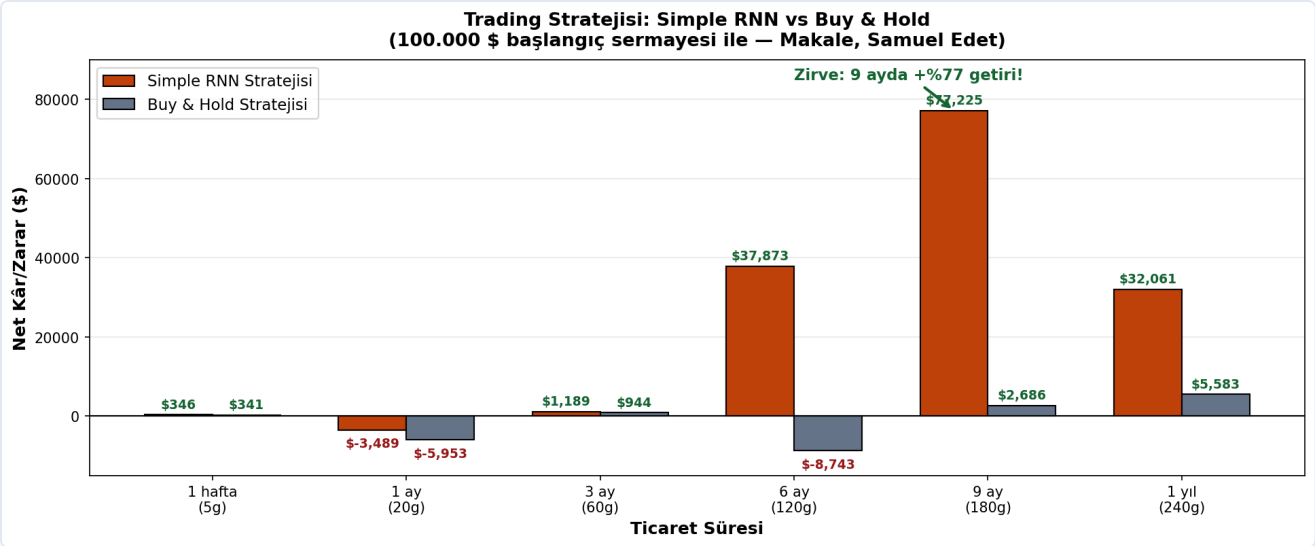
Ama uyarı: Makalenin test seti **yalnızca 99 gün**. Bu, istatistiksel olarak benim 47.276 günlük test setimden çok daha küçük bir örneklem. Yani "%75 doğruluk" bir miktar da *şanstan* kaynaklanıyor olabilir.

5.4 Trading Model: Tahminden Gerçek Kâra

Makalenin en çarpıcı katkısı, tahminleri gerçek bir **al-sat stratejisine** dönüştürmesidir. Strateji basit:

- Model "yükseliş (1)" derse: Elimdeki tüm nakitle S&P 500 alırım.
- Model "düşüş (0)" derse: Elimdeki tüm hisseleri satıp nakde dönerim.

Bu strateji, klasik **Buy & Hold (Al ve Tut)** stratejisi ile karşılaştırılıyor. Buy & Hold, "ilk gün al, son gün sat, arada hiçbir şey yapma" şeklindeki en temel yaklaşımdır.



Şekil 7. *Simple RNN stratejisi vs Buy & Hold stratejisi: 100.000 $ başlangıç sermayesi ile altı farklı trading süresi boyunca elde edilen net kâr/zarar. RNN stratejisinin 9 aylık +%77 getirisi dikkat çekici.*

9 ayda %77 getiri — yani 100.000 dolar, 9 ayda 177.000 dolar olmuş. Buy & Hold aynı sürede sadece 2.685 dolar kazandırmış. Elbette bu hesaba işlem maliyetleri dahil değil ve küçük bir örneklem üzerinde yapılan simülasyon, ama **tahmin başarısının gerçek para kazancına dönüşebileceğini kanıtlıyor.**

6. Sonuç ve İleriye Dönük Çıkarımlar

Bu karşılaştırma bana şunu net olarak öğretti: Benim projem **bir hedef değil, bir başlangıç**. Makine öğrenmesinde buna **"baseline model"** — temel karşılaştırma modeli — denir.

Neyi doğru yaptım?

- ✓ **Leakage-safe pipeline:** `shuffle=False` ve train-only scaling ile zamansal bütünlüğü korudum.
- ✓ **Overfitting önlemleri:** Dropout + Early Stopping ile modelin ezberlemesini engelledim.
- ✓ **Zengin özellik seti:** 9 teknik gösterge ile tek bir ham fiyattan çok daha fazla bilgi ürettim.
- ✓ **Dev veri seti:** 257.000 satırlık örneklem sayesinde sonuçlar istatistiksel olarak anlamlı.

Nerede eksik kaldım?

- ✗ **Hafızasız model:** Feedforward, zaman serisinin kalbindeki ardışık bağımlılığı yakalayamıyor.
- ✗ **Tek bir mimari:** Deney tasarımı olmadan "bu en iyi mimari" demek imkânsız.
- ✗ **Sınıf dengesizliği:** `class_weight` kullanmadığım için model çoğunluk sınıfına meyilli.

Bir sonraki adım — Sunumumda da belirttiğim gibi:

1. **RNN / LSTM / GRU mimarilerine geçmek.** Bu, %55'ten %75'e sıçramanın en büyük tetikleyicisi olacaktır. Finansal zaman serisi, doğası gereği zamansal bir yapıdır — onu zamansal bir modelle işlemek zorundayız.
2. **Sliding Window (Kayan Pencere) tekniği uygulamak.** Bu, tek bir günün özelliklerini kullanmak yerine, "son 10 günün özelliklerini bir pencere halinde topla, bir sonraki günün yönünü tahmin et" demektir. Veri yapısı 2D'den 3D'ye (*Örneklem* × *Zaman Adımı* × *Özellik*) dönüşür. Tıpkı bir doktorun hastayı tek bir muayenede değil, son haftaki tahlillerle beraber değerlendirmesi gibi.
3. **Attention / Transformer mimarilerini denemek.** Bunlar RNN'in modern evrimidir; uzun vadeli bağımlılıkları LSTM'den daha iyi yakalar. GPT gibi büyük dil modellerinin altında yatan teknolojidir.
4. **Class weight ve threshold tuning.** Sınıf dengesizliğini düzelttikten sonra karar eşiğini 0.50'den 0.55'e çekerek "daha az ama daha kaliteli tahminler" üretmek.

Son Söz

MIT 6.S191 dersinde öğretilen Perceptron → Dense → Multi-layer zinciri, benim projemde gerçek bir finansal problemin sert sınırlarıyla buluştu. Ve o sınırlar bana şunu gösterdi: **Derin öğrenme sihirli bir değnek değil — doğru aracı, doğru probleme uygulama sanatıdır.** Finansal zaman serileri için o doğru araç, Dense değil; LSTM'dir.

Samuel Edet'in makalesi, benim bir sonraki hedefimin haritasıdır. Yolun başındayım ama yol net. Feedforward ile %55'e geldim; LSTM ve Sliding Window ile %70'lerin kapısını çalmak artık teknik bir mesele, mimari bir gizem değil.

Ve belki en önemli ders: **Makine öğrenmesinde başarı, sadece modelin güzelliğinde değil, veriye olan saygıda yatar.** `shuffle=False` koymak, bazen 200 epoch eğitmekten daha kıymetlidir.